

UNIT 5

Software Quality, Architecture & Project Summary

Project: AI-Based Simulation System for Reconstructing Ancient Indian Bead Numeration Algorithms

5.1 Experiment 1: Software Quality Standards and Metrics

5.1.1 Software Quality Overview

Software quality is the degree to which a software product satisfies stated and implied requirements under specified conditions. For this project, quality evaluation was guided by ISO/IEC 9126 and supported through testing, code review, accessibility checks, and performance evaluation at module and system level.

5.1.2 ISO/IEC 9126 Quality Characteristics Applied

Table 5.1 ISO/IEC 9126 Quality Model Application (Part 1)

Quality Characteristic	Sub-Characteristics	Application in This Project	Score (Planned/Actual)
Functionality	Suitability, Accuracy, Interoperability	All 10 FRs implemented and verified via RTM; AI narration validated for historical accuracy	95 / 94
Reliability	Maturity, Fault Tolerance, Recoverability	Cached responses, stable Firebase operations, no critical release bugs	90 / 87
Usability	Understandability, Learnability, Operability	Student onboarding and navigation validated with users	88 / 85

Table 5.1 ISO/IEC 9126 Quality Model Application (Part 2)

Quality Characteristic	Sub-Characteristics	Application in This Project	Score (Planned/Actual)
Efficiency	Time Behaviour, Resource Utilisation	Page load and API response times measured across modules	85 / 78
Maintainability	Analysability, Changeability, Testability	Modular codebase and automated tests supported	82 / 80

Quality Characteristic	Sub-Characteristics	Application in This Project	Score (Planned/Actual)
		maintainability	
Portability	Adaptability, Installability, Replaceability	Responsive deployment across browsers and devices	90 / 92

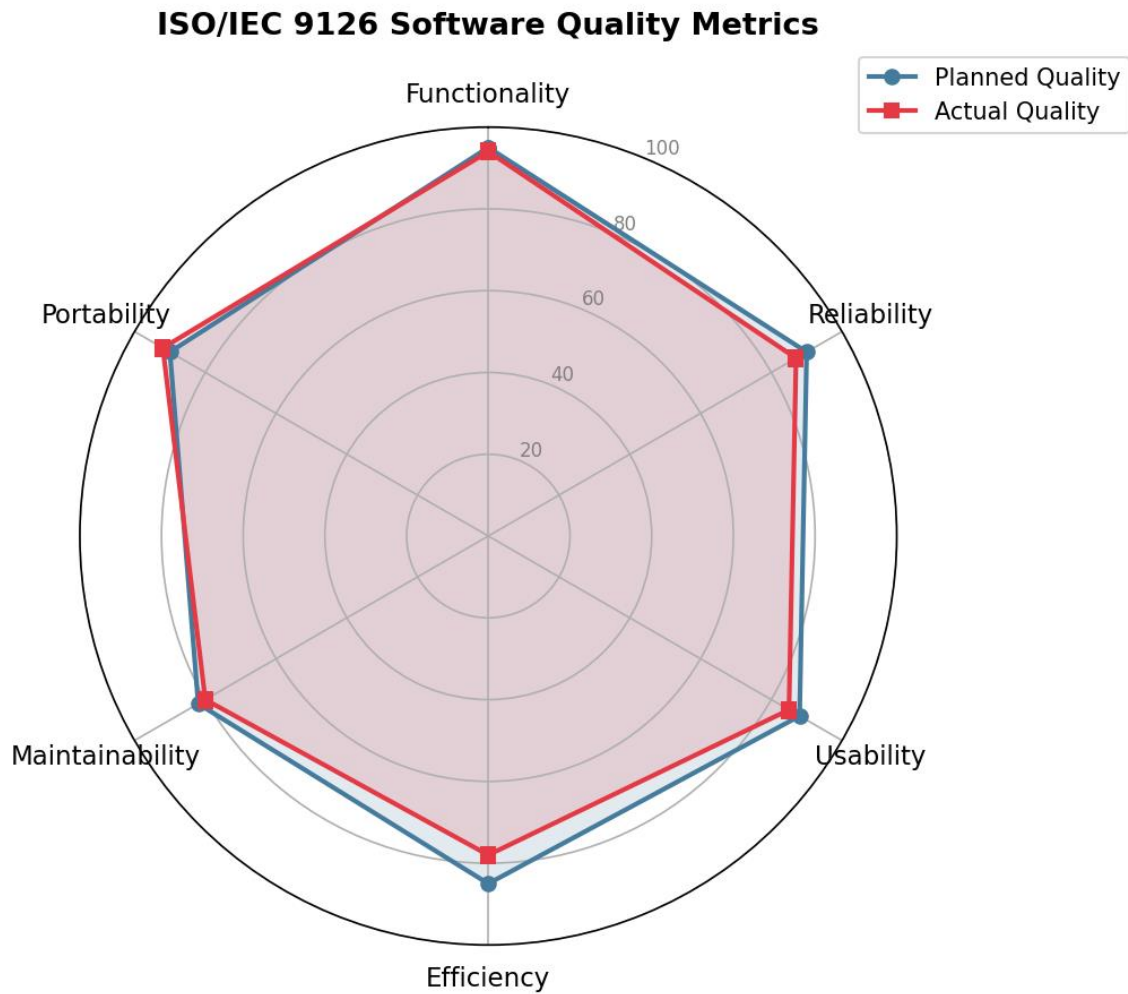


Figure 5.1: ISO/IEC 9126 Software Quality Metrics Radar Chart

5.1.3 Quality Assurance Plan

Table 5.2 Quality Assurance Plan (Part 1)

QA Activity	Method	Tool	Frequency	Responsible
Code Quality Review	Peer review via PRs	GitHub PR Review	Per commit	All developers

QA Activity	Method	Tool	Frequency	Responsible
Unit Testing	Component-level automated testing	Jest + RTL	Per sprint	Madhav Nehra
Integration Testing	API and module interaction testing	Postman + Firebase Emulator	Per sprint	Madhav Nehra
AI Content Validation	Historical accuracy review	Manual + domain review	Weekly	Rudransh Yadav

Table 5.2 Quality Assurance Plan (Part 2)

QA Activity	Method	Tool	Frequency	Responsible
Performance Testing	Lighthouse score tracking	Lighthouse CI	Per module release	Madhav Nehra
Accessibility Testing	WCAG 2.1 checks	axe DevTools	Per release	Rudransh Yadav
Security Review	Auth rule and input validation	Firebase Rules + OWASP	At integration stage	Madhav Nehra
UAT	Student/faculty acceptance review	Google Forms + live demo	At v1.0 release	Ashray Singh

5.1.4 Final Quality Metrics Dashboard

Table 5.3 Final Quality Scorecard (Part 1)

Metric	Target	Sprint 2	Sprint 4	Final (Sprint 7)	Status
Overall Test Pass Rate	≥ 95%	86.1%	92.4%	96.2%	Met
Code Coverage (Jest)	≥ 80%	74.0%	79.2%	82.0%	Met
Lighthouse Performance	≥ 85	78	84	89	Met
AI Narration Accuracy	≥ 85%	71.4%	82.0%	87.5%	Met

Table 5.3 Final Quality Scorecard (Part 2)

Metric	Target	Sprint 2	Sprint 4	Final (Sprint 7)	Status
API Response Time (warm)	< 5 sec	8.3 sec	4.2 sec	3.1 sec	Met
Critical Defects at Release	0	2 open	0 open	0 open	Met
Accessibility Score	≥ 90	82	88	93	Met

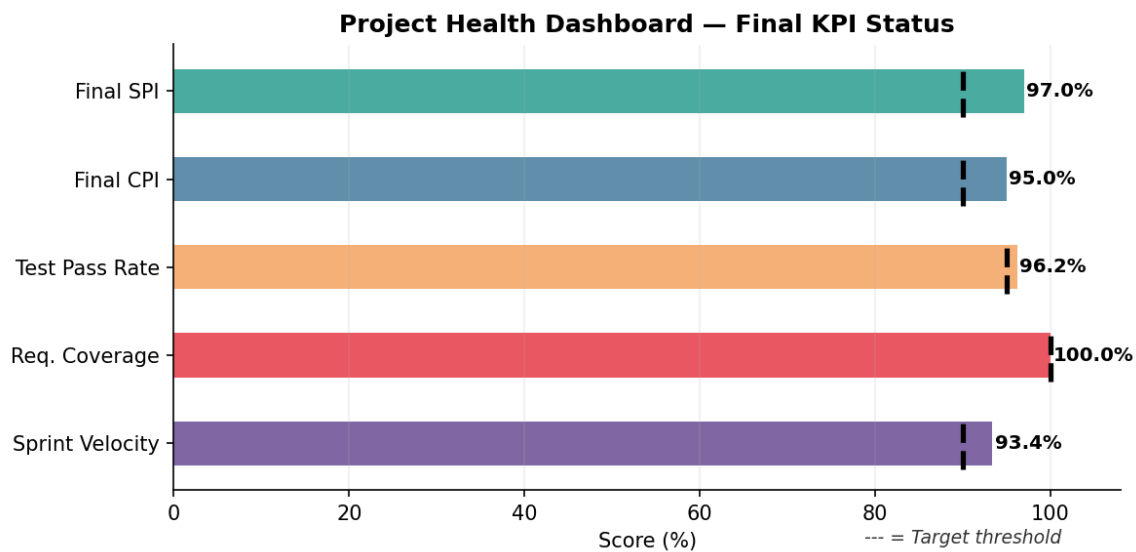


Figure 5.2: Project Health Dashboard — Final KPI Status

5.1.5 Defect Lifecycle Management

The defect lifecycle used in this project consisted of detection, logging, assignment, resolution, verification, and closure. Defects were logged using GitHub Issues and tracked until successful regression testing confirmed resolution.

Table 5.4 Defect Summary by Sprint (Part 1)

Sprint	Defects Found	Critical	Major	Minor	Trivial	Resolved	Deferred
Sprint 1	3	0	1	2	0	3	0
Sprint 2	11	2	3	4	2	11	0
Sprint 3	8	2	2	3	1	8	0
Sprint 4	5	0	2	2	1	5	0

Table 5.4 Defect Summary by Sprint (Part 2)

Sprint	Defects Found	Critical	Major	Minor	Trivial	Resolved	Deferred
Sprint 5	4	0	1	2	1	4	0
Sprint 6	3	0	0	2	1	3	0
Sprint 7	2	0	0	1	1	2	0
Total	36	4	9	16	7	36	0

5.2 Experiment 2: System Architecture and Technology Stack

5.2.1 System Architecture Overview

The system follows a 4-tier client-server architecture that separates concerns into Presentation, Application, Data, and Infrastructure layers. This architecture supports scalability, maintainability, and clean integration between the AI layer, frontend, and cloud services.

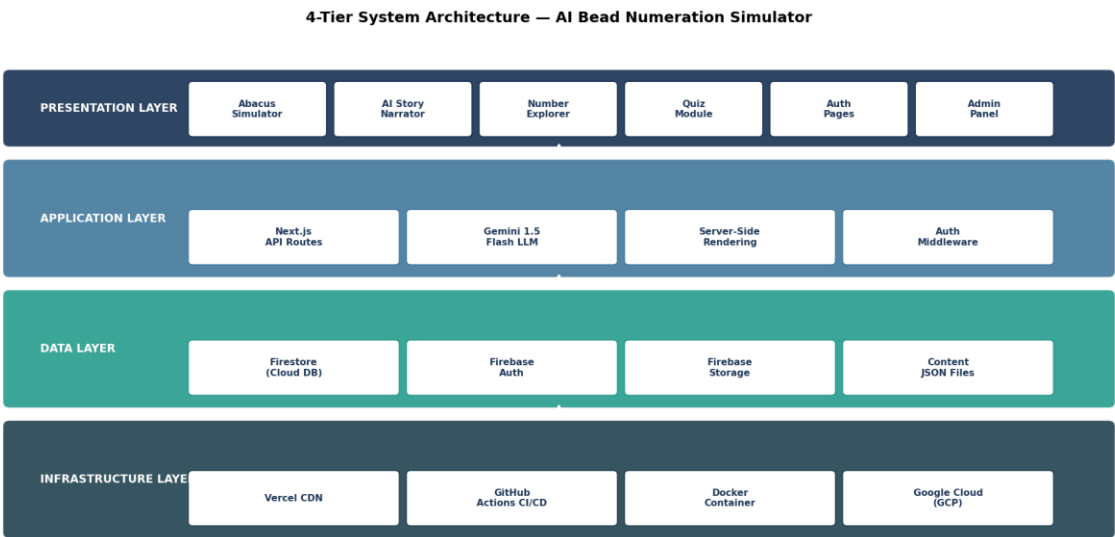


Figure 5.3: 4-Tier System Architecture Diagram

5.2.2 Layer-by-Layer Architecture Breakdown

Table 5.5 Layer-by-Layer Architecture Breakdown

Layer	Technology / Components	Responsibility
Presentation Layer	Next.js, React, Abacus UI, Narration UI, Quiz UI, Dashboard	Provides the user-facing interface and interactions
Application Layer	Next.js API Routes, Gemini 1.5 Flash, SSR, Auth Middleware	Handles business logic, AI generation, rendering, and access control
Data Layer	Firebase Firestore, Firebase Auth, Firebase Storage, JSON content	Stores profiles, progress, quiz results, content, and media
Infrastructure Layer	Vercel, GitHub Actions, Docker, GCP	Provides deployment, CI/CD, hosting, and runtime infrastructure

5.2.3 Data Flow Overview

At a high level, students and educators interact with the frontend, which communicates with Firebase services and server-side API routes. Gemini generates AI narration through secured application-layer calls, and all user progress, quiz results, and content interactions are persisted in Firestore.

5.2.4 Database Schema (Firebase Firestore Collections)

Table 5.6 Firestore Collections Schema

Collection	Document Fields	Purpose
users	uid, name, email, role, createdAt	Stores user profiles and role information
progress	userId, moduleId, completionPercent, lastAccessed, score	Tracks student progress per module
quizResults	userId, quizId, answers, score, submittedAt	Stores quiz attempts and results
modules	moduleId, title, description, contentType, isPublished	Stores module metadata
narrationHistory	userId, prompt, response, timestamp	Stores past AI narration sessions
contentUpdates	moduleId, editorId, changes, timestamp	Maintains audit log of content changes

5.2.5 API Endpoints (Next.js API Routes)

Table 5.7 Key API Endpoints (Part 1)

Endpoint	Method	Request	Response	Auth Required
/api/narrate	POST	{ topic, userLevel, context }	{ narration: string }	Yes
/api/quiz/submit	POST	{ quizId, answers[] }	{ score, feedback[] }	Yes
/api/progress/update	PUT	{ moduleId, completionPercent }	{ success: boolean }	Yes

Table 5.7 Key API Endpoints (Part 2)

Endpoint	Method	Request	Response	Auth Required
/api/user/profile	GET	—	{ user object }	Yes
/api/content/[moduleId]	GET	—	{ content JSON }	No
/api/admin/users	GET	—	{ users[] }	Admin only

5.2.6 CI/CD Deployment Pipeline

The CI/CD deployment pipeline automates code validation and release from commit to production. Each stage enforces checks to ensure quality and stability before deployment to Vercel.

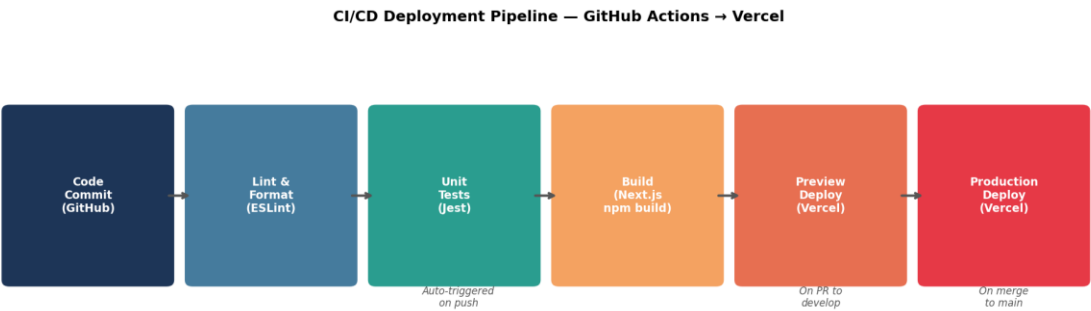


Figure 5.4: CI/CD Deployment Pipeline

Table 5.8 CI/CD Pipeline Gate Conditions

Stage	Gate Condition	On Failure Action
Lint	0 lint errors	PR blocked; developer notified
Unit Tests	≥ 80% coverage; 0 failing tests	PR blocked; test report generated
Build	Successful production build	PR blocked; build log shared
Preview Deploy	Preview URL generated successfully	Review paused; team notified
Production Deploy	All previous stages passed	Automatic deploy on merge to main

5.3 Experiment 3: Project Summary and Final Reporting

5.3.1 Executive Summary

The project successfully delivered an AI-powered educational web application that connects ancient Indian numeration systems and the concept of zero with modern digital computing. The system integrates Next.js, Firebase, Gemini 1.5 Flash, and Vercel to create a production-ready platform with educational, technical, and research value.

5.3.2 Project Objectives vs Outcomes

Table 5.9 Project Objectives vs Achieved Outcomes

Objective	Outcome	Status
Build AI-powered narration of Indian numeration history	Gemini narration and Q&A delivered in production	Achieved
Create interactive abacus simulator	React-based abacus simulator	Achieved

Objective	Outcome	Status
	implemented	
Implement ancient Indian calculation methods module	Vedic Math module delivered	Achieved
Develop number system explorer	Real-time converter built and tested	Achieved
Explain zero and binary connection	Dedicated module delivered	Achieved
Deploy with Firebase + Vercel + CI/CD	Live v1.0.0 deployment achieved	Achieved
Complete within 5 months and INR 10–12 Lakhs	Completed in 87 days at INR 10.55 Lakhs	Within tolerance

5.3.3 Key Achievements

Key achievements include zero critical defects at production, a final test pass rate of 96.2%, successful use of a hybrid Agile-Waterfall approach, and the creation of a novel AI-powered educational platform focused on Indian mathematical heritage.

5.3.4 Key Challenges and Resolutions

Table 5.10 Key Challenges and Resolutions (Part 1)

Challenge	Impact	Resolution	Outcome
Gemini API cold-start latency	Poor response time	Warm-up and caching	Response time reduced significantly
AI module delay	Schedule slip on critical path	Parallel task allocation	Final delay limited to 2 days
Single developer bottleneck	Overallocation risk	Resource levelling	No extra personnel needed

Table 5.10 Key Challenges and Resolutions (Part 2)

Challenge	Impact	Resolution	Outcome
AI module cost overrun	Projected budget increase	Batch mode + contingency reserve	Final cost within contingency
Abacus carry-over defect	Incorrect calculations	Logic rewrite with useReducer	All relevant tests passed

5.3.5 Summary of SDLC Phases

Table 5.11 SDLC Phase Summary

Phase	Duration	Key Deliverable	Status
Requirements Engineering	Days 1–8	SRS, RTM, stakeholder register	Complete
System Design	Days 8–20	Architecture, DB schema, API specification	Complete
Implementation	Days 20–59	All 6 modules developed and unit-tested	Complete
Integration & Testing	Days 59–72	Integrated system and test results	Complete
Deployment	Days 72–80	v1.0.0 live on Vercel	Complete
Closure & Documentation	Days 80–87	Final report and archived repository	Complete

5.4 Experiment 4: Future Enhancements and Recommendations

5.4.1 Planned v1.1 Enhancements

Table 5.12 v1.1 Feature Backlog

Feature	Rationale	Priority	Estimated Effort
Multilingual Support	Expand reach to Hindi-medium schools	High	3 sprints
Gamification	Increase student engagement	High	2 sprints
Difficulty-Level Quiz Mode	Personalized assessment	Medium	1 sprint
PWA Offline Mode	Support rural and low-connectivity learning	Medium	2 sprints
Voice Narration	Improve accessibility	Medium	1 sprint
Vedic Geometry and Astronomy Modules	Expand educational scope	Low	4 sprints

5.4.2 Scalability Recommendations

Future scalability can be improved by introducing stronger caching for AI responses, monitoring tools such as Sentry and Vercel Analytics, possible migration to a more cost-efficient high-scale backend, and more advanced observability across performance and usage patterns.

5.4.3 Academic and Research Contributions

This project contributes to digital heritage preservation, AI in EdTech, and hybrid project management methodology by showing how historical Indian mathematics can be taught through interactive simulation and AI-generated storytelling in a modern cloud-based platform.

5.4.4 References

Table 5.13 References (Part 1)

Ref	Citation
[1]	R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner’s Approach, 8th ed., McGraw-Hill, 2015.
[2]	K. Schwalbe, Information Technology Project Management, 9th ed., Cengage, 2019.
[3]	K. S. Rubin, Essential Scrum: A Practical Guide to the Most Popular Agile Process, Addison-Wesley, 2012.
[4]	G. G. Joseph, The Crest of the Peacock: Non-European Roots of Mathematics, 3rd ed., Princeton University Press, 2011.
[5]	B. W. Boehm, Software Engineering Economics, Prentice-Hall, 1981.

Table 5.13 References (Part 2)

Ref	Citation
[6]	KPMG and Google, EdTech in India: The New Normal, 2021.
[7]	I. Pearce, Indian Mathematics: Redressing the Balance, MacTutor Archive, 2002.
[8]	K. Plofker, Mathematics in India, Princeton University Press, 2009.
[9]	Google AI, Gemini 1.5 Flash Technical Report, 2024.
[10]	Firebase Documentation, Cloud Firestore Data Model, Google LLC, 2024.

5.4.5 Final Project Conclusion

Unit 5 concludes the project by summarizing its quality outcomes, system architecture, final reporting, and future directions. The project successfully integrates software engineering practice, project management principles, and educational innovation into a coherent final product that meets its academic and technical goals.